

RecoverAI — Forensic Data Recovery Platform

COMPLETE TECHNICAL SPECIFICATION & EVALUATOR Q&A; GUIDE

CalmStacks Hackathon 2026 · Malnad College of Engineering (MCE Hassan)

Live Demo: <https://recovery.onrender.com> | GitHub: github.com/samhithm1-crypto/recovery

1. Project Overview & Problem Statement

RecoverAI is a full-stack, AI-augmented digital forensics and data recovery suite. Modern data recovery tools (e.g., TestDisk, Scalpel, PhotoRec) are historically confined to dense command-line interfaces, demanding expert knowledge in hexadecimal manipulation, file system internals, and manual signature carving. When storage media is corrupted, formatted, or files are emptied from system trash, general users and investigators face steep hurdles.

RecoverAI bridges this gap by combining **low-level binary signature scanning**, **mathematical Shannon entropy analysis**, **Windows file system metadata reverse-engineering (\$Recycle.Bin parsing)**, and **LLM-driven forensic reasoning (Google Gemini)** into an intuitive, responsive web portal. The system empowers investigators and everyday users to inspect, reconstruct, restore, export, and securely email recovered artifacts.

2. Full Technology Stack & Architectural Taxonomy

2.1 Frontend Engineering (Vanilla Web Standards)

Technology	Specification	Architectural Purpose & Implementation Detail
HTML5	Living Standard	Semantic document layout, forensic status panels, data tables, accessible modal dialogs.
CSS3 (Vanilla)	Flexbox / Grid / Glassmorphism	Dark forensic HUD theme (#0d1117/#dc2626 accents), backdrop filters, CSS variables, mobile media queries.
Vanilla JavaScript	ES2022+ Native Async/Await	Zero framework overhead (no React/Vue bloat). Direct DOM diffing, async Fetch API handlers, client state management.
HTML5 Canvas API	2D Rendering Context	Dynamic background particle animation representing binary data stream fragments interacting on screen.
jsPDF & AutoTable	v2.5.1 / v3.8.2	Client-side vector PDF generation for instantaneous forensic report exports without server roundtrip latency.
Dynamic Origin API	Custom Base URL Resolver	Auto-detects localhost:5000 vs. live Render cloud origin (window.location.origin) for seamless CI/CD execution.

2.2 Backend & Forensic Core (Python & WSGI)

Module / Tool	Type	Role & Implementation Details
Python 3.11+	Runtime	High-level execution environment for rapid binary byte processing and API serving.
Flask	Micro-framework	Lightweight, un-opinionated WSGI routing server delivering RESTful endpoints and serving static assets.
Gunicorn	WSGI HTTP Server	Production-grade UNIX pre-fork worker pool for concurrency, timeout management, and Render stability.
Google Gemini AI	Generative LLM SDK	Synthesizes structural forensic narratives, evaluates data damage severity, guides reconstruction tactics.
smtplib & email.mime	Standard Library	RFC 821 SMTP client over TLS/SSL (port 465) for secure dispatch of forensic PDFs and recovered binary payloads.
struct & ctypes	Low-level Binary C interop	Unpacks Little-Endian binary structs (\$I recycle metadata, Windows FILETIME stamps, 64-bit offsets).
hashlib	Cryptographic Engine	Calculates SHA-256 digests over all carved fragments to guarantee legal chain-of-custody data integrity.
shutil & os	Filesystem Engine	Safe file extraction, collision avoidance, and Desktop destination directory staging.

3. System Architecture & Information Pipeline

The RecoverAI architecture is segregated into three functional tiers: **Client Presentation & Local Processing**, **Python Forensic Intelligence Engine**, and **External Cloud Services**.

[CLIENT TIER: Browser / Mobile] • UI Rendering (HTML5/CSS3 HUD) Canvas Data Particles jsPDF Vector Engine • Async Fetch Controller (Uploads binary payloads, parses JSON telemetry, executes downloads)
▼ RESTful HTTP / JSON over HTTPS (Port 443 / Dev 5000)
[APPLICATION & FORENSIC TIER: Flask + Gunicorn WSGI] • app.py: Route Controller, Multipart File Buffer, SMTP Delivery Hub • analyzer.py: Magic Byte Scanner (18 formats) Shannon Entropy Calculator SHA-256 Hasher • deleted_recovery.py: Windows \$Recycle.Bin (\$I/\$R) Binary Parser Drive Free-Space Raw Carving
▼ Outbound External API & Service Channels
[EXTERNAL CLOUD SERVICES] • Google Gemini 3.1 Flash: Forensic Assessment & Reconstruction Guidance via Generative AI • Gmail SMTP (SSL 465): Multi-part MIME forensic report and recovered file dispatch to user inbox • Render Cloud PaaS: Containerized build, environment secret injection, and auto-deploy pipeline

4. Core Module Implementation Deep Dive

4.1 analyzer.py — The Forensic Analysis & Carving Engine

- **Magic Byte Pattern Matcher:** Reads input streams and searches for magic byte signatures at variable offsets. Supports 18 formats across Images (JPEG, PNG, GIF), Documents (PDF, DOC/OLE), Media (MP3, MP4, WAV), Archives (ZIP, 7Z, GZ, BZ2), Databases (SQLite), and Executables (PE/ELF).
- **Shannon Entropy Scoring:** Evaluates byte frequency distribution over 0–255 to measure algorithmic disorder. Used to flag encryption, compression, or corruption.

- **Integrity Classification Algorithm:** Combines entropy deviation, header-trailer completeness, and expected file bounds into a normalized 0–100% score (RECOVERABLE ≥75%, PARTIAL 40–74%, CRITICAL <40%).
- **Cryptographic Chain of Custody:** Automatically streams data through hashlib.sha256(), returning a non-repudiable hash string.

4.2 deleted_recovery.py — Native Windows Trash & Drive Carving

- **Recycle Bin Reverse Engineering:** Iterates through all available system drive letters via Win32 GetLogicalDrives(). Enters \$Recycle.Bin SID folders. Parses Windows 10/11 \$I index records using struct.unpack('<QQQ') for header version, original file byte size, and 64-bit Windows FILETIME (converted to UTC epoch). Reads the original absolute path string (UTF-16LE).
- **Non-Destructive Restoration:** Matches the \$I metadata file to its twin \$R content file. Restores the file to Desktop/RecoverAI_Restored/ using shutil.copy2() with automatic collision numbering.
- **Direct Email Dispatch:** Once restored, the file can be immediately emailed to the user as an authenticated SMTP attachment without requiring manual browsing.

4.3 app.py — RESTful API Gateway & Orchestration

- Enforces CORS sanity and serves the frontend as single-page web assets.
- Implements base64 stream decoding for client-generated PDF attachments and dispatches MIME emails via Gmail SMTP SSL.

5. Forensic Science: Shannon Entropy Formulation

Claude Shannon's information theory entropy formula calculates the average information density per symbol in a byte sequence:

$$H = - \sum [p(x) \cdot \log_2(p(x))] \quad (where\ p(x)\ is\ the\ occurrence\ frequency\ of\ byte\ x \in [0,\ 255])$$

Entropy Range (H)	Underlying Data Characteristic	Forensic Classification & Recoverability
0.00 – 1.99	Sparse, repeating, or zero-padded sectors	Unallocated / wiped blocks. Low forensic value.
2.00 – 4.99	Plaintext (ASCII/UTF-8), source code, logfiles	High recoverability; structural text intact.
5.00 – 6.99	Structured binary formats, compiled bytecode, bitmaps	Normal binary structures; candidate for carving.
7.00 – 8.00	Compressed files (ZIP/GZ) or Encrypted blocks (AES)	Max entropy. If headers corrupted, unrecoverable without keys.

6. Supported File Signatures (Magic Bytes)

Format	Hex Signature (Magic Bytes)	Offset	Category
.jpg / .jpeg	FF D8 FF	0	Raster Image
.png	89 50 4E 47 0D 0A 1A 0A	0	Lossless Image
.pdf	25 50 44 46 (ASCII: %PDF)	0	Document
.zip / .docx	50 4B 03 04 (ASCII: PK..)	0	Archive / Office
.sqlite / .db	53 51 4C 69 74 65 (ASCII: SQLite)	0	Relational DB
.mp4 / .mov	00 00 00 .. 66 74 79 70 (ftyp)	4	Digital Video
.mp3	49 44 33 (ID3v2) or FF FB	0	Digital Audio
.exe / .dll	4D 5A (MZ DOS Header)	0	PE Executable
.7z	37 7A BC AF 27 1C	0	High-ratio Archive
.gif	47 49 46 38 37 61 / 39 61	0	Graphic Image

7. Complete Evaluator Q&A; Preparation Guide

This section compiles comprehensive, defensible responses to every technical question hackathon evaluators or cybersecurity judges are expected to ask.

Q1: What is the core innovation of RecoverAI compared to established tools like Autopsy, TestDisk, or Recuva?

Most forensic tools are heavy desktop-only applications with steep learning curves, producing cryptic hexadecimal logs that require a certified examiner to interpret. RecoverAI innovates by combining binary carving with an accessible web-native UI and Generative AI (Gemini). The AI acts as a virtual forensic assistant, translating raw entropy and structural metadata into actionable recovery narratives, while providing one-click recovery and automated secure email dispatch directly from the browser.

Q2: How does magic byte carving work when files are fragmented or non-contiguous on disk?

File carving scans sector by sector for known header signatures. In contiguous files, carving reads from header to known EOF trailer (e.g., FFD9 for JPEG). When files are fragmented across non-contiguous clusters, traditional carving encounters corrupted bodies. RecoverAI combats this by combining header identification with Shannon entropy anomaly detection (which flags sudden shifts in byte distribution indicating a boundary transition) and fragment relationship clustering based on cluster offset proximity.

Q3: How does the Google Gemini AI integration actually assist in recovery? Does it hallucinate binary data?

RecoverAI never asks Gemini to invent raw binary bytes. Instead, Gemini is provided with structured forensic metadata: fragment magic types, offset intervals, calculated entropy, byte integrity scores, and header/trailer presence flags. Gemini evaluates the structural damage, explains what parts of the file are intact, provides recovery confidence ratings, and recommends forensic actions (e.g., hex patch strategies, container re-wrapping). It acts as a forensic reasoning layer, not an arbitrary byte generator.

Q4: Explain the Windows Recycle Bin structure and how RecoverAI reads 'deleted' files.

In modern Windows (NTFS), deleting a file to the Recycle Bin moves it into a hidden folder 'C:\\$Recycle.Bin\'. Windows renames the file into two paired files: '\$Ixxxxxx' and '\$Rxxxxxx'. The '\$I' file is a 544-byte binary index containing header version, 64-bit original size, 64-bit deletion FILETIME, and UTF-16 original file path. The '\$R' file contains the actual raw payload. RecoverAI parses the binary '\$I' structure directly using Python's struct module, maps it to '\$R', and restores it to Desktop without invoking Windows GUI components.

Q5: What is the significance of computing SHA-256 for each carved fragment?

In digital forensics, legal evidence must satisfy the 'Chain of Custody' standard. Computing a SHA-256 cryptographic hash over carved fragments at the exact moment of extraction ensures data integrity. If a recovered artifact is produced in legal or audit proceedings, matching SHA-256 digests prove that the file was not altered, fabricated, or contaminated by the investigator or the recovery software.

Q6: How does the client-side PDF export work, and why was it chosen over server-side PDF generation?

Client-side PDF generation is handled via jsPDF and jsPDF-AutoTable directly inside app.js. This architecture reduces server CPU overhead on our free Render cloud tier, eliminates server-side PDF rendering dependencies (like WeasyPrint or wkhtmltopdf), and provides instantaneous exports to the user. For the email feature, the browser simply converts the generated vector PDF blob into base64 and posts it to Flask for SMTP relay.

Q7: How does RecoverAI handle CORS and API environments between local testing and production?

RecoverAI features a self-configuring API URL resolver: `'const API = (hostname === 'localhost' || hostname === '127.0.0.1') ? 'http://localhost:5000/api' : window.location.origin + '/api'`. Since Flask serves both the static web HUD and the REST API from the same origin on Render, all requests are same-origin, eliminating cross-origin preflight latency and CORS blocking issues while allowing local development on port 5000.

Q8: Why did you choose pure Vanilla JS, CSS, and HTML instead of React, Next.js, or TailwindCSS?

Speed, footprint, and maintainability. A forensic recovery utility must load instantly with zero overhead. React/Next.js introduce substantial node_modules dependencies, hydrate delays, and complex build steps. By engineering Vanilla ES2022 and modern CSS with custom properties and glassmorphism, our total frontend weighs less than 120KB, executes at 60 FPS, has zero npm vulnerabilities, and runs natively in any browser.

Q9: How are credentials and API keys safeguarded in the application?

All sensitive credentials—including GEMINI_API_KEY, SMTP_EMAIL, and SMTP_PASSWORD—are strictly isolated in the '.env' file and excluded from version control via '.gitignore'. On Render, credentials are injected as secure environment variables, with 'SMTP_PASSWORD' explicitly flagged as non-syncing. Furthermore, Gmail delivery uses a scoped 16-character Google App Password rather than personal account credentials.

Q10: What are the current operational limitations of RecoverAI?

Current constraints include: (1) Free Render hosting instances sleep after 15 minutes of inactivity, resulting in a cold-start delay on first request; (2) Drive raw carving and Recycle Bin extraction are Windows-specific due to NTFS structure and Win32 drive APIs; (3) Browser upload limits restrict disk image uploads to reasonable sizes (<100MB) on the cloud deployment; and (4) Severely fragmented files without headers cannot be reconstructed without deep dictionary carving.

Q11: How would you scale RecoverAI into an enterprise forensic platform?

To scale for enterprise investigations, we would: (1) Implement a Celery / Redis asynchronous task queue to process multi-gigabyte disk dumps in the background; (2) Stream disk images directly to Amazon S3 or GCP Cloud Storage with pre-signed URLs; (3) Implement multi-tenant authentication via OAuth2/JWT; (4) Add Linux ext4 and Apple APFS inode forensic parsers; and (5) Package as an Electron desktop app with raw kernel drive access.

Q12: What is the difference between Flask's built-in dev server and Gunicorn in production?

Flask's built-in 'app.run()' server is single-threaded, synchronous, and intended strictly for local development; it easily blocks on heavy I/O and crashes under concurrent traffic. Gunicorn (Green Unicorn) is an industrial WSGI HTTP server implementing a pre-fork worker model that distributes incoming requests across multiple Python worker processes, automatically handles process recycling, and prevents connection drops on cloud platforms like Render.

8. Project Deliverable Metrics

Metric	Count / Specification	Architectural Relevance
Python Codebase	~900 lines (app.py, analyzer.py, deleted_recovery.py)	Compact, zero-redundancy backend forensic pipeline
Frontend Codebase	~2,100 lines JS + 873 lines CSS	High-performance responsive HUD, Canvas particle system
REST Endpoints	8 Active Endpoints	Covers analysis, reconstruction, bin recovery, restore, and email
File Signatures	18 Standard Signatures	Images, Documents, Audio, Video, Archives, Databases, Executables
Features Delivered	20 Complete Features	All hackathon objectives met + verified live in production
Live URL	https://recoverai.onrender.com	Publicly accessible for live judges testing